

Technical Walkthrough: Simulated Mini-SOC Pipeline

Name: Austin Kobayashi

1. Design Rationale

This capstone is a small simulated SOC pipeline that shows how AI can support detection and response work without touching real systems. The project uses AI generated synthetic telemetry in the shape of Suricata eve.json records. It does not use live traffic, real malware, production firewall rules, endpoint isolation, or a production SIEM or SOAR platform.

This was a purposeful design choice, as the prototype needs to be realistic enough to explain a SOC workflow, but it should still be safe and manageable in scope. A local dataset, local outputs, and simulated response actions demonstrate the process without creating risk for an actual network.

The pipeline also separates the decision into several stages. An event is ingested, checked, turned into features, classified, scored, reviewed by deterministic safety checks, routed to a response path, and then preserved in evidence logs. That structure matters because a model prediction by itself is not enough to justify an automated response. If an event is malformed, evasive, or uncertain, the workflow sends it to review instead of trusting the classifier by default.

The model is a scikit-learn decision tree because it is a simple, bounded, and explainable classifier. A larger model might be more impressive, but it would also be harder to explain in a short capstone. Here the goal is to show a semi autonomous logic layer that can classify events, assign risk, choose safe simulated actions, and leave behind a clear decision trail. The model is trained in memory on each run from the combined synthetic dataset, so there is no saved model artifact. The evidence log records that honestly with `model_sha256: "in_memory_not_persisted"`.

The response logic is intentionally modest. The system can close low risk benign events, create JSON tickets, send uncertain cases to a review queue, and write malicious source IPs to a simulated blocklist text file. The blocklist is only a local artifact. It does not connect to a firewall or change any real network control.

2. Architecture Diagram

The implemented workflow follows this structure:

```
AI generated synthetic Suricata style events
-> Event intake and validation
-> Feature extraction
-> Decision tree classifier
-> Risk scoring
-> Deterministic safety checks
-> Response routing
-> Evidence decision log
-> Forensic export package
```

Safety and response branches:

```
-> Human review queue  
-> Simulated incident ticket  
-> Simulated IP blocklist
```

The main pipeline command is:

```
python -m minisoc.cli respond data\sample\demo_scenario_events_20.eve.jsonl --train  
data\raw\synthetic_minisoc_events_combined_2200.eve.jsonl --outputs outputs
```

That command trains the in memory classifier, processes the demo events, applies safety checks, writes response artifacts, and creates the decision log. A separate export command packages evidence for one selected event:

```
python -m minisoc.cli export-evidence --event-id demo-0008 --events  
data\sample\demo_scenario_events_20.eve.jsonl --decisions  
outputs\decision_logs\decisions.jsonl --outputs outputs
```

In the verified demo run, the pipeline processed 20 demo records. It closed 5 events, created 8 tickets, routed 3 events to review, and wrote 4 entries to the simulated blocklist. Those counts are useful because they show that the system does not flatten every alert into the same outcome.

3. Model Or Logic Explanation

The first stage is event intake. The intake code reads line delimited JSON records that resemble Suricata eve.json events. It validates required fields such as event_id, timestamp, event_type, src_ip, dest_ip, asset_id, expected_label, and the synthetic ground truth disclosure. It also calculates a SHA-256 hash of the raw JSON line and carries it into the decision log as evidence metadata.

After intake, feature extraction turns each event into numeric signals that the classifier and risk scorer can use. These signals include alert severity, destination port, failed login count, internal probe count, callback interval, domain entropy, URI entropy, byte and packet ratios, asset criticality, missing field flags, obfuscation flags, and flags for text that resembles prompt injection. The feature extractor also produces reason codes, so the output is still human readable.

The classifier is a bounded decision tree trained from the combined synthetic dataset. It predicts one of six labels: benign, credential_access, lateral_movement, command_and_control, exploit_attempt, or needs_human_review. The classification result includes the predicted label, confidence score, model name, model version, feature names, and reason codes.

Risk scoring runs after classification. It combines model confidence, alert severity, asset criticality, suspicious activity patterns, and suspicious input indicators into a score from 0 to 100. For example, a credential access event with many failed logins receives a higher score than normal benign traffic. Incomplete or evasive input also raises the score because uncertainty is itself useful information for a SOC analyst.

The deterministic safety checks sit between scoring and response routing. This review gate blocks automation when confidence is low, when the classifier predicts needs_human_review, when intake

metadata requires review, when fields are missing, when obfuscation indicators are present, when log text resembles prompt injection, or when a benign prediction conflicts with a high risk score. The classifier helps interpret the event, but the review gate decides whether automation is allowed.

Response routing chooses the next safe action based on the score, label, and safety check result. Low risk benign events can be closed, medium and high risk events can create tickets, and suspicious or unclear events go to the review queue. High scoring command and control or exploit activity can write the source IP to the simulated blocklist when automation is allowed.

4. Bypass And Failure Handling

The demo dataset includes suspicious edge cases so the pipeline shows more than a clean happy path, including records with missing analysis fields, obfuscated values, high entropy strings, and text in a log field that resembles prompt injection. The system treats those values as event data rather than instructions.

When an event is malformed or incomplete, intake and feature extraction mark the missing fields, risk scoring raises the priority, and the review gate blocks automation. Obfuscated values follow a similar path: the feature extractor counts the indicators, the risk score increases, and the event goes to human review. If a log field contains one of the simple instruction-like marker phrases, the pipeline flags the record and sends it to review instead of treating that text as trusted context.

That behavior matters because a real SOC pipeline has to handle incomplete logs, hidden indicators, and conflicting signals from different security tools. In those cases, this prototype uses a safe default by preserving the event, recording the reason for concern, and requiring analyst review before any action is taken.

5. Forensic Readiness Strategy

The pipeline preserves evidence throughout the workflow so a reviewer can trace how a synthetic event moved from intake to response. At intake, the raw JSON line is hashed with SHA-256 before the event is normalized. The response workflow carries that hash into `outputs/decision_logs/decisions.jsonl`, along with the event identity, correlation details, model output, scoring result, safety check outcome, selected simulated action, and analyst review fields. That record gives an analyst the main facts needed to explain why the event was closed, queued for review, turned into a ticket, or added to the simulated blocklist.

The local response artifacts support the same review trail from an operational point of view. Events that need analyst judgment are written to `outputs/review_queue/review_queue.jsonl`, ticketed events are saved under `outputs/tickets/`, and simulated block decisions are appended to `outputs/simulated_blocklist.txt`. These files are intentionally simple because the project is a local mini-SOC demonstration, but together they show whether the pipeline treated the event as benign, uncertain, suspicious, or serious enough for a simulated containment step.

The `export-evidence` command creates a small review package for one selected event under `outputs/evidence_exports/<event_id>/`. Instead of asking a reviewer to piece together the event from several output folders, the export places the raw event, decision record, model metadata, action record, analyst summary, and hash manifest in one location. The hash manifest records SHA-256 values for the exported files, which gives a basic integrity check if the package is reviewed later.

This is not a real chain of custody system, and it should not be presented as one. It is a local forensic readiness demonstration that preserves the information an analyst would need for incident response or audit review: what the original event looked like, how the model classified it, how the score and safety checks affected the decision, what simulated action was taken, and whether the exported evidence changed after it was created.

6. Known Limitations

This prototype is intentionally narrow in scope. The telemetry is AI generated synthetic data, so the evaluation should be read as a check that the model learned the patterns built into the scenario, not as proof that it would perform well in a real SOC. The project also stays fully local. It does not ingest live traffic, run Suricata, parse real PCAPs, process malware, or connect to production security tools.

The response layer follows the same controlled approach. A simulated block is written to a text file, tickets are saved as JSON artifacts, and the review queue is stored as JSONL. That lets the demo show how events would move through different response paths without changing a firewall rule, opening a real service desk ticket, or pretending to be an analyst console.

The AI component is also kept simple on purpose. The classifier is a decision tree trained in memory from `data\raw\synthetic_minisoc_events_combined_2200.eve.jsonl` each time the command runs. There is no persisted model artifact, drift monitoring, retraining pipeline, or live feedback loop from analyst decisions. The baseline version also avoids using an LLM, which reduces prompt handling risk but limits natural language enrichment to the generated analyst summary in the export package.

Taken together, these limits are part of the design rather than hidden production gaps. The project is a safe, explainable mini-SOC prototype that connects event intake, threat classification, risk scoring, review checks, simulated response, evidence logging, and forensic export in one traceable workflow. It is not production ready, but it shows how a security automation pipeline can leave enough evidence behind for a reviewer to understand what happened and why.

Integrity Statement

I confirm that this submission is my original work. Any external tools, references, datasets, frameworks, or code libraries used in this project have been acknowledged. I understand that the work must represent my own design, analysis, and implementation decisions.